Campus Event Management System

A

Skill Oriented Course Web Application / Project report submitted in the partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

<<Student_Name>> <<Roll_Number>>

Under the esteemed guidance of

Mr. N. SrinivasaRao

Sr. Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAKIREDDY BALIREDDY COLLEGE OF ENGINEERING (AUTONOMOUS)

Accredited by NAAC with 'A' Grade & NBA (Under Tier - I),

An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution

Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada

L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P. - 521230.

2024-2028

LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING (AUTONOMOUS)

Accredited by NAAC with 'A' Grade & NBA (Under Tier - I),

An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution

Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada

L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P. - 521230.

Department of

COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Skill Oriented Course Web Application / Project entitled "Campus Event Management System" is being submitted by

<<Student_Name>> <<Roll_Number>>

in partial fulfillment of the requirements for the award of degree of B.Tech in Computer Science and Engineering from Jawaharlal Nehru Technological University Kakinada is a record of Bonafide work carried out by him/her at Lakireddy Bali Reddy College of Engineering (Autonomous).

The results embodied in this Skill Oriented Course Web Application / Project report have not been submitted to any other university or institute for the award of any degree or diploma.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to thank Mr. N. SrinivasaRao, Sr. Assistant Professor, CSE department for the encouragement and support in carrying out this Skill Oriented Course Web Application / Project.

I also take the privilege to record my thanks to Dr. S. Nagarjuna Reddy, Professor, Head of the Department of CSE whose encouragement, cooperation, and valuable support crown my success.

I express my thanks to the support given by management in completing my Web Application / Project. I also express my sincere gratitude & deep sense of respect to Dr.K. Appa Rao, Principal for making us available all the required assistance and his support and inspiration to carry out this Skill Oriented Course Web Application / Project in the Institute.

I am thankful to the teaching and non-teaching staff of the CSE department for their direct as well as indirect help in my Skill Oriented Course Web Application / Project.

I am elated to avail myself to this opportunity to express my deep sense of gratitude to my parents.

<<Name of the Student>> <<Roll_Number>>

DECLARATION

I am here to declare that the project entitled “Campus Event Management System” work done by me. I certify that the work contained in the report is original and has been done by me under the guidance of my supervisor. The work has not been submitted to any other institute in preparing for any degree or diploma. I have followed the guidelines provided by the institute for preparing the report. I have confirmed the norms and guidelines given in the Ethical Code of Conduct of the Institute. Whenever I have used materials (data, theoretical analysis, figures and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owner of the sources, whenever necessary.

Signature of the student

<<Student Name>> <<Roll Number>>

ABSTRACT

Campus Event Management System is a full stack Java web application that centralizes campus event publication, participant registration, organizer administration, and registration reporting. It replaces disconnected notices and manual lists with a database-backed portal. The application provides two roles, Participant and Organizer, with separate dashboards and permissions.

Participants register using a unique roll number, email address, and phone number. Email ownership is verified using a time-limited one-time password before account submission. Passwords are stored using BCrypt with a cost factor of 12. Participants browse event cards, filter by category and department, inspect event descriptions, and register for eligible events. Each completed registration receives a unique pass ID and a confirmation email. Registered events and participant profile information are available in separate sidebar sections.

Organizers authenticate with their organizer ID and password followed by an email OTP. They create, update, view, and delete events, upload event images, define registration deadlines and capacities, and inspect registration counts. Searchable reports support event, college, department, and study-year filters. The application uses HTML, CSS, JavaScript, JSP, Servlets, JDBC, MySQL, and Apache Tomcat 9. HttpSession maintains authenticated state between login and logout. Payment is simulated in the current academic implementation and is recorded as PAID; no live payment gateway is integrated.

LIST OF CONTENTS

CONTENTS **PAGE NO**

INTRODUCTION 8

Overview of the Project 8

Feasibility Study 9

TECHNOLOGIES USED 10

HTML, CSS & JavaScript (or) React JS 10

JDBC (Oracle or Mongo DB) 11

Servlets (or) Node JS & Express JS 12

JSP 13

WEB APPLICATION ARCHITECTURE 14

CODING & IMPLEMENTATION 15

RESULTS 78

CONCLUSION 80

REFERENCES 81

LIST OF ABBREVIATIONS

HTML - Hypertext Markup Language

CSS - Cascading Style Sheets

JS - JavaScript

JDBC - Java Database Connectivity

JSP - JavaServer Pages

JDK - Java Development Kit

JVM - Java Virtual Machine

SQL - Structured Query Language

DBMS - Database Management System

HTTP - Hypertext Transfer Protocol

SMTP - Simple Mail Transfer Protocol

OTP - One Time Password

CRUD - Create Read Update Delete

DAO - Data Access Object

MVC - Model View Controller

WAR - Web Application Archive

TLS - Transport Layer Security

BLOB - Binary Large Object

INTRODUCTION

Overview of the Project

Campus activities include technical competitions, workshops, sports tournaments, and cultural programmes. When announcements, registration forms, and participant lists are maintained separately, students may miss deadlines and organizers may have inconsistent records. The Campus Event Management System addresses this coordination problem through a single web application backed by a relational database.

The main objective is to provide verified participant onboarding, convenient event discovery, controlled event registration, and accurate organizer reports. The system identifies students through their roll numbers and prevents duplicate email addresses and phone numbers. Organizer accounts are provisioned by initialization rather than through public registration.

The application has two authenticated roles. Participants can browse available events, inspect details, register, retrieve a unique pass, and view their profile. Organizers maintain events, registration limits, deadlines, images, and availability, and inspect participant reports. These role boundaries are enforced in Servlet operations in addition to the authenticated route filter.

The participant registration form includes text inputs, password and confirmation inputs, an address text area, college and department dropdowns, a four-year dropdown, gender radio buttons, profile image upload, a show-password checkbox, Submit, and Reset. An email OTP must be verified before registration is submitted; the server also checks the verified email state so client-side controls are not the only protection.

Each participant dashboard contains Dashboard, Available Events, Registered Events, and Profile sections. Summary cards link to event lists. Event cards contain an image, category, department, date, venue, seats, fee, and action buttons. Descriptions are displayed only on the event details page. Both event lists support category and department filtering.

Register now creates a registration only when the event is active, its deadline has not passed, capacity is available, and the participant has not registered already. Successful registration opens Registered Events and displays a confirmation message. Already Registered appears in green, and a dialog provides participant information, payment state, registration timestamp, and unique pass ID.

The organizer dashboard contains Dashboard, All Events, Registration Reports, and Profile. A New Event action is available from the sidebar. Organizers can perform all event CRUD operations, inspect counts, filter events by category and department, and open members for a selected event. The report screen supports search and event, college, department, and year filters.

The project intentionally excludes participant cancellation and feedback. It records registration payment as PAID for demonstration. A production payment gateway, self-service password recovery, and administrative organizer management are possible later additions rather than current capabilities.

Feasibility Study

Technical Feasibility

The required technologies are supported by the current project: Java 17, Maven, JSP, Servlets, JDBC, MySQL, and Tomcat 9. The application uses javax.servlet and therefore targets Tomcat 9. A conventional browser provides the client interface, while JavaMail connects to Gmail SMTP for live email delivery. Local development needs a running MySQL service and an available Tomcat HTTP port.

Operational Feasibility

Participants use familiar form controls and clearly named sidebar sections. Registration and OTP status messages guide onboarding. Organizers use one event form and a report table rather than maintaining separate spreadsheets. Standard college, department, category, and year dropdown values improve consistency during entry and filtering.

Economic Feasibility

The academic solution can be developed with existing college computers and commonly available development tools. It does not require a paid payment service because payment is simulated. Deployment costs depend on future hosting, SMTP usage, database resources, and operational support; the project does not claim a measured cost saving.

Schedule Feasibility

The layered design supports incremental implementation: schema and models, authentication, participant onboarding, event CRUD, event registration, reports, and interface verification. Each module has a limited responsibility, allowing problems to be isolated during development and demonstration.

Data Feasibility

The three core tables represent users, events, and registrations. Primary keys, unique indexes, and foreign keys support consistent identities and relationships. Student and event photos are stored as BLOBs, avoiding filesystem synchronization requirements for the small deployment.

Security and Deployment Feasibility

BCrypt cost 12, session-based authentication, OTP verification, role checks, and prepared statements provide the implemented security foundation. Production readiness would additionally require secret management, rate limiting, CSRF protection, secure HTTPS deployment, and automated tests. The classroom application should be evaluated within that scope.

TECHNOLOGIES USED

The project uses the Java web technology option represented by this template. HTML, CSS, and JavaScript provide the browser interface; JDBC connects to MySQL; Servlets handle requests; and JSP renders views. The alternative framework and database names retained in the contents headings are template labels. React, Node.js, Express, Oracle, and MongoDB are not used by this implementation.

HTML, CSS & JavaScript (or) React JS

HTML

HTML structures the login, participant registration, dashboard, event, and report pages. Semantic forms define required fields and submit requests to clean Servlet endpoints. Input types support passwords, email, phone, dates, time, numbers, and file uploads. Dropdowns restrict standard values, while dialog elements display pass details.

CSS

The stylesheet controls spacing, typography, colours, card grids, sidebars, buttons, forms, dialogs, and tables. Responsive rules adjust the layout for smaller screens. The visual design is intentionally simple and professional, with clear contrast and no animation. Success states distinguish registrations that are already complete.

JavaScript

JavaScript validates fields before submission, manages password visibility, displays OTP status, enables submission after successful email verification, controls registration dialogs, and applies event date constraints. Server-side validation repeats critical rules because browser scripts can be bypassed.

Form Validation

Passwords require 6 to 15 characters with a lowercase letter, uppercase letter, number, and special symbol. Confirmation must match. Study year is restricted to 1 through 4. Departments are CSE, IT, AI&DS, CSM, EEE, ECE, MECH, CIVIL, and ASE. College names come from the fixed ten-college list shared with server validation.

Browser Interaction

Participant and organizer sections are reached through /dashboard, /events, /registrations, /reports, and /profile. The browser does not expose JSP filenames as application endpoints. After a successful event registration, a redirect to /registrations follows the POST request so refreshing the list does not repeat the submitted registration.

JDBC (Oracle or Mongo DB)

JDBC is the Java API used for relational database access. This application uses MySQL Connector/J rather than Oracle or MongoDB. Dao.getConnection obtains a connection using the configured JDBC URL, username, and password. AppDao contains schema creation, participant lookups, registration uniqueness checks, event operations, image access, and report queries.

PreparedStatement binds request values as parameters. This avoids constructing SQL by concatenating participant names, search text, or other submitted data. ResultSet rows are mapped to User, Event, ParticipantEventRegistration, and RegistrationRecord model objects before they are supplied to the views.

The users table stores roll_no, full_name, email, phone_number, college_name, password_hash, role, department, study_year, gender, address, profile_photo, profile_photo_type, and created_at. The roll number is the primary key; email and phone have unique constraints. Organizer-specific optional student attributes can remain empty.

The events table stores id, title, description, category, department, venue, event_date, registration_closes_at, capacity, fee, image_data, image_type, organizer_id, active, and created_at. The organizer foreign key connects each event to a user account. Event images are streamed from database BLOBs through EventImageServlet.

The registrations table stores id, event_id, participant_id, pass_id, payment_status, and registered_at. A unique constraint on event_id and participant_id prevents repeated registration for one participant and event. pass_id is also unique. Event deletion cascades to its registration records.

The relationship between participants and events is many to many through registrations. One organizer can create many events, and each event can contain many registrations. Registration logic checks availability and uses database constraints and transaction handling to protect consistency. Connections, statements, and result sets are closed with try-with-resources.

Application initialization creates tables when they do not exist, ensures required event columns and phone uniqueness, and inserts the organizer when absent. Database connection values can be supplied through DB_URL, DB_USER, and DB_PASSWORD. Startup failures and database errors are logged for diagnosis.

Servlets (or) Node JS & Express JS

Java Servlets form the controller layer; Node.js and Express are not used. Annotation mappings assign clean URL paths to the controllers. doGet generally loads a view or image, while doPost processes form submissions. Controllers obtain session identity, validate request parameters, call AppDao or EmailService, and forward to a JSP or redirect to a route.

Login accepts the account identifier and password and verifies the BCrypt hash. A valid participant can proceed directly to a dashboard. A valid organizer receives an email OTP, and OtpServlet completes the pending login only after verification. LogoutServlet invalidates the session.

RegisterServlet accepts multipart participant registration, validates mandatory values, checks verified email ownership, enforces uniqueness, reads the profile image, hashes the password, and saves the user. RegistrationEmailOtpServlet sends and verifies the email code and stores temporary verification state in the session.

DashboardServlet provides the role-specific dashboard and selects the available, registered, or profile section according to the endpoint. It applies category and department filters to the appropriate event list. EventServlet loads details and accepts organizer create, update, and delete operations, including image uploads and date validation.

RegistrationServlet registers participants for events, creates pass information through the DAO, sends the confirmation email, and redirects successful requests to Registered Events. ReportServlet supplies organizer reports with search and dropdown filters. EventImageServlet and ProfilePhotoServlet stream image bytes with their stored content types.

AuthFilter protects authenticated routes. The application stores the authenticated User in HttpSession and sets cache-control headers on protected responses. web.xml configures a 30-minute session timeout, an HttpOnly session cookie, and cookie-based session tracking. AppContextListener initializes the database at startup.

The project is packaged with mvn package as target/campus-events.war and deployed under the campus-events context on Tomcat 9. The HTTP port is chosen in the Tomcat or Eclipse configuration. MySQL and SMTP connectivity must be available for the corresponding workflows.

JSP

JavaServer Pages provide the server-rendered view layer. JSP pages receive request attributes prepared by a Servlet and use the authenticated session user to present role-specific information. The views are stored under WEB-INF/views so the application exposes controller routes rather than direct JSP filenames.

login.jsp displays the login form, password visibility control, Reset, Login, and participant registration link. register.jsp contains the complete participant form, email OTP controls, password requirements, show-password checkbox, profile image upload, Submit, and Reset. verify-otp.jsp completes organizer authentication.

userdashboard.jsp implements the participant sidebar and sections. Available Events and Registered Events each use their own GET filter form. Registration dialogs display name, roll number, email, phone, pass ID, payment status, and registration timestamp. Profile includes the photograph and all participant details.

organizerdashboard.jsp implements organizer summaries, all-event cards, filters, registration reports, and profile. event-form.jsp renders the organizer event form or participant event detail view according to the role. Back links preserve the intended event-list destination.

Model objects separate retrieved data from page layout. Event carries the event details and registration count. User represents account and profile data. ParticipantEventRegistration combines an event and its pass information, and RegistrationRecord represents organizer report rows.

The UI follows a simple hierarchy with consistent buttons, field labels, error messages, and sidebar states. Descriptions remain on the details screen rather than crowding cards. Uploaded images are loaded through protected endpoints. Required fields and dropdowns reduce incomplete and inconsistent data entry.

WEB APPLICATION ARCHITECTURE

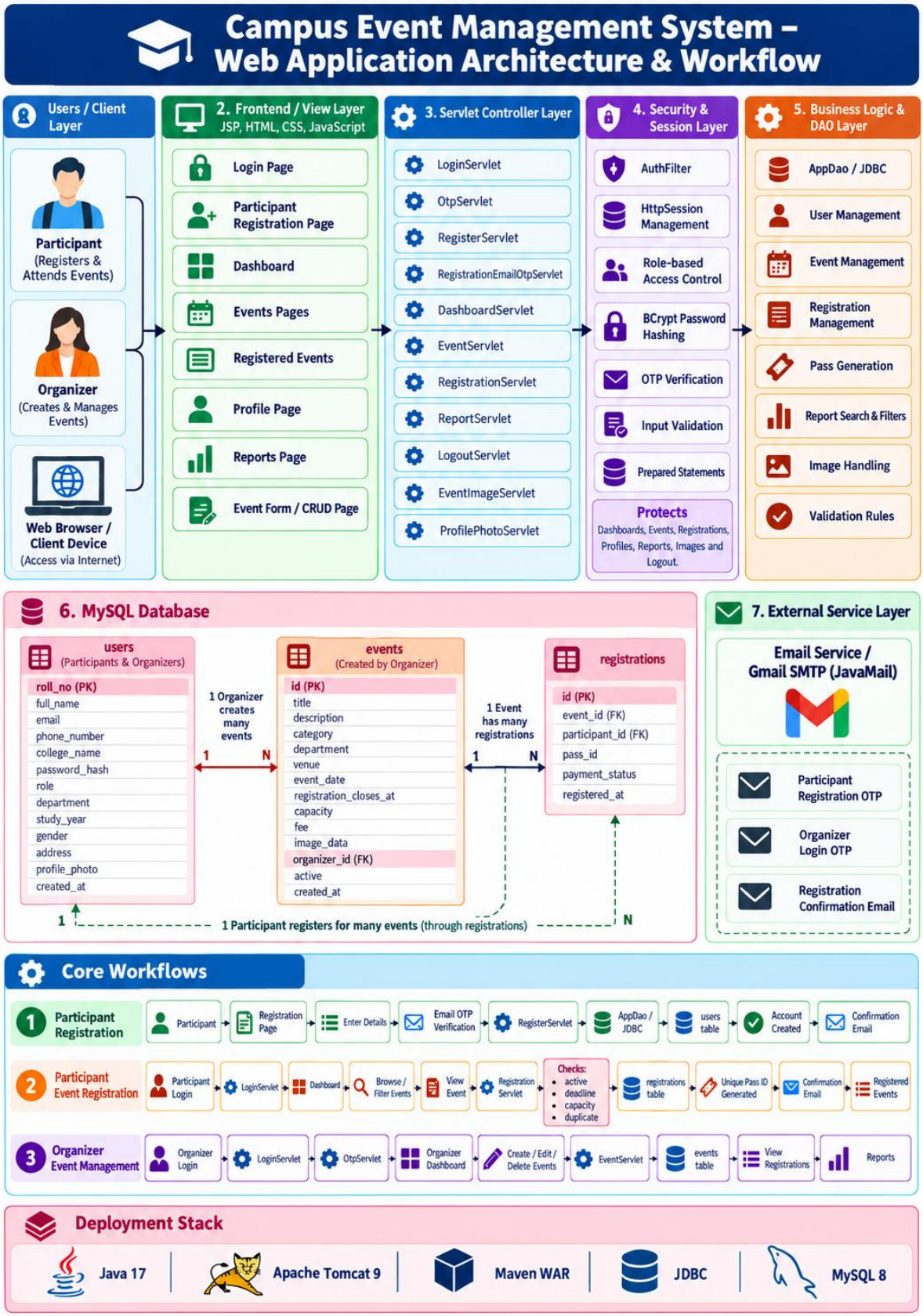


Figure 1 Campus Event Management System architecture and workflow

CODING & IMPLEMENTATION

The following listings contain every Java source file in the current src/main/java directory. Each listing includes the filename and package path and preserves the source order and indentation. The SMTP app password and database password literal are masked in this report; the working application source has not been altered. The project uses Java 17, javax.servlet, MySQL Connector/J, JavaMail, and jBCrypt.

The implementation is organized into com.model for data objects, com.dao for JDBC operations and schema initialization, com.util for password hashing, com.service for email, and com.servlets for request handling, authentication, images, reports, and startup. Each file starts on a separate page for clear identification.

1 AppDao.java

src/main/java/com/dao/AppDao.java

```
package com.dao;

import com.model.Event;
import com.model.ParticipantEventRegistration;
import com.model.RegistrationRecord;
import com.model.User;
import com.util.PasswordUtil;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.UUID;

public class AppDao {

    private static final String USERS = "users";
    private static final String EVENTS = "events";
    private static final String REGISTRATIONS = "registrations";

    public void initialize() throws SQLException {
        try (Connection connection = Dao.getConnection();
             Statement statement = connection.createStatement()) {

            statement.executeUpdate(
                "CREATE TABLE IF NOT EXISTS " + USERS + " ("
                    + "roll_no VARCHAR(30) PRIMARY KEY,"
                    + "full_name VARCHAR(100) NOT NULL,"
                    + "email VARCHAR(150) NOT NULL UNIQUE,"
                    + "phone_number VARCHAR(15),"
                    + "college_name VARCHAR(150),"
                    + "password_hash VARCHAR(255) NOT NULL,"
                    + "role ENUM('PARTICIPANT','ORGANIZER') NOT NULL,"
                    + "department VARCHAR(10),"
                    + "study_year INT,"
                    + "gender VARCHAR(20),"
                    + "address VARCHAR(500),"
                    + "profile_photo MEDIUMBLOB,"
                    + "profile_photo_type VARCHAR(100),"
                    + "created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,"
                    + "CONSTRAINT uk_users_phone UNIQUE(phone_number)"
                    + ")");
```

```
            statement.executeUpdate(
                    "CREATE TABLE IF NOT EXISTS " + EVENTS + " ("
                            + "id INT PRIMARY KEY AUTO_INCREMENT,"
                            + "title VARCHAR(150) NOT NULL,"
                            + "description TEXT NOT NULL,"
                            + "category VARCHAR(60) NOT NULL,"
                            + "department VARCHAR(10) NOT NULL,"
                            + "venue VARCHAR(150) NOT NULL,"
                            + "event_date DATETIME NOT NULL,"
                            + "registration_closes_at DATETIME NOT NULL,"
                            + "capacity INT NOT NULL,"
                            + "fee DECIMAL(10,2) DEFAULT 0,"
                            + "image_data MEDIUMBLOB,"
                            + "image_type VARCHAR(100),"
                            + "organizer_id VARCHAR(30) NOT NULL,"
                            + "active BOOLEAN DEFAULT TRUE,"
                            + "created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,"
                            + "CONSTRAINT fk_cem_event_organizer "
                            + "FOREIGN KEY(organizer_id) REFERENCES " + USERS + "(roll_no)"
                            + ")");

            statement.executeUpdate(
                    "CREATE TABLE IF NOT EXISTS " + REGISTRATIONS + " ("
                            + "id INT PRIMARY KEY AUTO_INCREMENT,"
                            + "event_id INT NOT NULL,"
                            + "participant_id VARCHAR(30) NOT NULL,"
                            + "pass_id VARCHAR(40) NOT NULL UNIQUE,"
                            + "payment_status ENUM('PAID','PENDING') DEFAULT 'PAID',"
                            + "registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,"
                            + "UNIQUE(event_id, participant_id),"
                            + "CONSTRAINT fk_cem_registration_event "
                            + "FOREIGN KEY(event_id) REFERENCES " + EVENTS
                            + "(id) ON DELETE CASCADE,"
                            + "CONSTRAINT fk_cem_registration_user "
                            + "FOREIGN KEY(participant_id) REFERENCES " + USERS + "(roll_no)"
                            + ")");
        }

        ensureUserConstraints();
        ensureEventColumns();
        seedOrganizer();
    }

    private void ensureUserConstraints() throws SQLException {
        try (Connection connection = Dao.getConnection();
                Statement statement = connection.createStatement()) {

            boolean phoneIndexExists = false;
            try (ResultSet indexes = connection.getMetaData().getIndexInfo(
                    connection.getCatalog(),
                    null,
```

```
                USERS,
                true,
                false)) {

            while (indexes.next()) {
                String columnName = indexes.getString("COLUMN_NAME");
                if ("phone_number".equalsIgnoreCase(columnName)) {
                    phoneIndexExists = true;
                    break;
                }
            }
        }

        if (!phoneIndexExists) {
            statement.executeUpdate(
                    "CREATE UNIQUE INDEX uk_users_phone ON "
                            + USERS + "(phone_number)");
        }
    }
}

private void ensureEventColumns() throws SQLException {
    try (Connection connection = Dao.getConnection();
            Statement statement = connection.createStatement()) {

        addColumnIfMissing(
                connection,
                statement,
                "department",
                "VARCHAR(10) NULL");
        addColumnIfMissing(
                connection,
                statement,
                "registration_closes_at",
                "DATETIME NULL");
        addColumnIfMissing(
                connection,
                statement,
                "image_data",
                "MEDIUMBLOB");
        addColumnIfMissing(
                connection,
                statement,
                "image_type",
                "VARCHAR(100)");

        statement.executeUpdate(
                "UPDATE " + EVENTS
                        + " SET registration_closes_at = event_date"
                        + " WHERE registration_closes_at IS NULL");
        statement.executeUpdate(
                "UPDATE " + EVENTS
```

```
                    + " SET department = 'CSE'"
                    + " WHERE department IS NULL OR department = "");
            statement.executeUpdate(
                "ALTER TABLE " + EVENTS
                    + " MODIFY COLUMN registration_closes_at DATETIME NOT NULL");
            statement.executeUpdate(
                "ALTER TABLE " + EVENTS
                    + " MODIFY COLUMN department VARCHAR(10) NOT NULL");

            dropColumnIfPresent(connection, statement, "image_url");
        }
    }

    private void addColumnIfMissing(
            Connection connection,
            Statement statement,
            String columnName,
            String definition) throws SQLException {

        try (ResultSet columns = connection.getMetaData().getColumns(
                connection.getCatalog(),
                null,
                EVENTS,
                columnName)) {

            if (!columns.next()) {
                statement.executeUpdate(
                    "ALTER TABLE " + EVENTS
                        + " ADD COLUMN " + columnName + " " + definition);
            }
        }
    }

    private void dropColumnIfPresent(
            Connection connection,
            Statement statement,
            String columnName) throws SQLException {

        try (ResultSet columns = connection.getMetaData().getColumns(
                connection.getCatalog(),
                null,
                EVENTS,
                columnName)) {

            if (columns.next()) {
                statement.executeUpdate(
                    "ALTER TABLE " + EVENTS + " DROP COLUMN " + columnName);
            }
        }
    }

    private void seedOrganizer() throws SQLException {
```

```
    String sql = "INSERT IGNORE INTO " + USERS
            + " (roll_no, full_name, email, password_hash, role) "
            + "VALUES (?, ?, ?, ?, 'ORGANIZER')";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, Dao.env("DEFAULT_ORGANIZER_ID", "ORG001"));
        statement.setString(2, "Campus Organizer");
        statement.setString(
                3,
                Dao.env("DEFAULT_ORGANIZER_EMAIL", "nitwg6@gmail.com"));
        statement.setString(
                4,
                PasswordUtil.hash(
                        Dao.env("DEFAULT_ORGANIZER_PASSWORD", "Admin@123")));
        statement.executeUpdate();
    }
}

public User findUser(String rollNumber) throws SQLException {
    String sql = "SELECT * FROM " + USERS + " WHERE roll_no = ?";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        try (ResultSet result = statement.executeQuery()) {
            return result.next() ? mapUser(result) : null;
        }
    }
}

public String registrationConflict(
        String rollNumber,
        String email,
        String phoneNumber) throws SQLException {

    String sql = "SELECT "
            + "COALESCE(MAX(roll_no = ?), 0) roll_exists, "
            + "COALESCE(MAX(email = ?), 0) email_exists, "
            + "COALESCE(MAX(phone_number = ?), 0) phone_exists "
            + "FROM " + USERS;

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        statement.setString(2, email);
        statement.setString(3, phoneNumber);

        try (ResultSet result = statement.executeQuery()) {
```

```
            if (!result.next()) {
                return null;
            }
            if (result.getBoolean("roll_exists")) {
                return "Roll number already exists.";
            }
            if (result.getBoolean("email_exists")) {
                return "Email address already exists.";
            }
            if (result.getBoolean("phone_exists")) {
                return "Phone number already exists.";
            }
            return null;
        }
    }
}

    public void createParticipant(
            User participant,
            String passwordHash,
            byte[] profilePhoto,
            String profilePhotoType) throws SQLException {

        String sql = "INSERT INTO " + USERS
                + " (roll_no, full_name, email, phone_number, college_name,"
                + " password_hash, role, department, study_year, gender, address,"
                + " profile_photo, profile_photo_type) "
                + "VALUES (?, ?, ?, ?, ?, ?, 'PARTICIPANT', ?, ?, ?, ?, ?, ?)";

        try (Connection connection = Dao.getConnection();
                PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setString(1, participant.rollNo());
            statement.setString(2, participant.fullName());
            statement.setString(3, participant.email());
            statement.setString(4, participant.phoneNumber());
            statement.setString(5, participant.collegeName());
            statement.setString(6, passwordHash);
            statement.setString(7, participant.department());
            statement.setInt(8, participant.year());
            statement.setString(9, participant.gender());
            statement.setString(10, participant.address());
            statement.setBytes(11, profilePhoto);
            statement.setString(12, profilePhotoType);
            statement.executeUpdate();
        }
    }

    public void deleteUser(String rollNumber) throws SQLException {
        String sql = "DELETE FROM " + USERS + " WHERE roll_no = ?";

        try (Connection connection = Dao.getConnection();
```

```
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        statement.executeUpdate();
    }
}

public List<Event> events(boolean activeOnly) throws SQLException {
    String sql = "SELECT e.*, "
            + "(SELECT COUNT(*) FROM " + REGISTRATIONS
            + " r WHERE r.event_id = e.id) registered_count "
            + "FROM " + EVENTS + " e"
            + (activeOnly
                    ? " WHERE e.active = TRUE"
                            + " AND e.event_date >= NOW()"
                            + " AND e.registration_closes_at >= NOW()"
                    : "")
            + " ORDER BY e.event_date";

    List<Event> events = new ArrayList<>();
    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql);
            ResultSet result = statement.executeQuery()) {

        while (result.next()) {
            events.add(mapEvent(result));
        }
    }
    return events;
}

public Event event(int eventId) throws SQLException {
    String sql = "SELECT e.*, "
            + "(SELECT COUNT(*) FROM " + REGISTRATIONS
            + " r WHERE r.event_id = e.id) registered_count "
            + "FROM " + EVENTS + " e WHERE e.id = ?";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setInt(1, eventId);
        try (ResultSet result = statement.executeQuery()) {
            return result.next() ? mapEvent(result) : null;
        }
    }
}

public Set<Integer> registeredEventIds(String rollNumber) throws SQLException {
    String sql = "SELECT event_id FROM " + REGISTRATIONS
            + " WHERE participant_id = ?";
    Set<Integer> eventIds = new HashSet<>();
```

```
    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        try (ResultSet result = statement.executeQuery()) {
            while (result.next()) {
                eventIds.add(result.getInt("event_id"));
            }
        }
    }
    return eventIds;
}

public List<ParticipantEventRegistration> participantRegistrations(String rollNumber)
        throws SQLException {

    String sql = "SELECT e.*, "
            + "(SELECT COUNT(*) FROM " + REGISTRATIONS
            + " counted WHERE counted.event_id = e.id) registered_count, "
            + "r.pass_id, r.payment_status, r.registered_at "
            + "FROM " + REGISTRATIONS + " r "
            + "JOIN " + EVENTS + " e ON e.id = r.event_id "
            + "WHERE r.participant_id = ? "
            + "ORDER BY e.event_date";

    List<ParticipantEventRegistration> registrations = new ArrayList<>();
    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        try (ResultSet result = statement.executeQuery()) {
            while (result.next()) {
                registrations.add(new ParticipantEventRegistration(
                        mapEvent(result),
                        result.getString("pass_id"),
                        result.getString("payment_status"),
                        result.getTimestamp("registered_at").toLocalDateTime()));
            }
        }
    }
    return registrations;
}

public void saveEvent(
        Event event,
        byte[] imageData,
        String imageType) throws SQLException {

    boolean creating = event.id() == 0;
    String sql = creating
            ? "INSERT INTO " + EVENTS
                    + " (title, description, category, department, venue, event_date,"
```

```
                + " registration_closes_at, capacity, fee, image_data, image_type,"
                + " organizer_id, active) "
                + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)"
            : "UPDATE " + EVENTS
                + " SET title = ?, description = ?, category = ?, department = ?,"
                + " venue = ?, event_date = ?, registration_closes_at = ?, capacity = ?,"
                + " fee = ?, image_data = COALESCE(?, image_data),"
                + " image_type = COALESCE(?, image_type),"
                + " organizer_id = ?, active = ? WHERE id = ?";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, event.title());
        statement.setString(2, event.description());
        statement.setString(3, event.category());
        statement.setString(4, event.department());
        statement.setString(5, event.venue());
        statement.setTimestamp(6, Timestamp.valueOf(event.date()));
        statement.setTimestamp(7, Timestamp.valueOf(event.registrationClosesAt()));
        statement.setInt(8, event.capacity());
        statement.setDouble(9, event.fee());
        statement.setBytes(10, imageData);
        statement.setString(11, imageType);
        statement.setString(12, event.organizerId());
        statement.setBoolean(13, event.active());
        if (!creating) {
            statement.setInt(14, event.id());
        }
        statement.executeUpdate();
    }
}

public byte[] eventImage(int eventId) throws SQLException {
    String sql = "SELECT image_data FROM " + EVENTS + " WHERE id = ?";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setInt(1, eventId);
        try (ResultSet result = statement.executeQuery()) {
            return result.next() ? result.getBytes("image_data") : null;
        }
    }
}

public String eventImageType(int eventId) throws SQLException {
    String sql = "SELECT image_type FROM " + EVENTS + " WHERE id = ?";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {
```

```
            statement.setInt(1, eventId);
            try (ResultSet result = statement.executeQuery()) {
                return result.next() ? result.getString("image_type") : null;
            }
        }
    }

    public void deleteEvent(int eventId) throws SQLException {
        String sql = "DELETE FROM " + EVENTS + " WHERE id = ?";

        try (Connection connection = Dao.getConnection();
                PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setInt(1, eventId);
            statement.executeUpdate();
        }
    }

    public String register(int eventId, String rollNumber) throws SQLException {
        String passId = "EVT-" + eventId + "-"
                + UUID.randomUUID().toString().substring(0, 8).toUpperCase();

        try (Connection connection = Dao.getConnection()) {
            connection.setAutoCommit(false);
            try {
                verifySeatAvailability(connection, eventId);
                insertRegistration(connection, eventId, rollNumber, passId);
                connection.commit();
                return passId;
            } catch (SQLException exception) {
                connection.rollback();
                throw exception;
            } finally {
                connection.setAutoCommit(true);
            }
        }
    }

    private void verifySeatAvailability(Connection connection, int eventId)
            throws SQLException {

        String sql = "SELECT capacity, "
                + "(SELECT COUNT(*) FROM " + REGISTRATIONS
                + " WHERE event_id = ?) used "
                + "FROM " + EVENTS
                + " WHERE id = ? AND active = TRUE"
                + " AND registration_closes_at >= NOW() FOR UPDATE";

        try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setInt(1, eventId);
            statement.setInt(2, eventId);
            try (ResultSet result = statement.executeQuery()) {
```

```
            if (!result.next() || result.getInt("used") >= result.getInt("capacity")) {
                throw new SQLException("Event is full or unavailable.");
            }
        }
    }
}

private void insertRegistration(
        Connection connection,
        int eventId,
        String rollNumber,
        String passId) throws SQLException {

    String sql = "INSERT INTO " + REGISTRATIONS
            + " (event_id, participant_id, pass_id, payment_status) "
            + "VALUES (?, ?, ?, 'PAID')";

    try (PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setInt(1, eventId);
        statement.setString(2, rollNumber);
        statement.setString(3, passId);
        statement.executeUpdate();
    }
}

public List<RegistrationRecord> report(
        Integer eventId,
        String collegeName,
        String department,
        Integer year,
        String search) throws SQLException {

    StringBuilder sql = new StringBuilder(
            "SELECT r.*, e.title, u.full_name, u.email, u.phone_number,"
                    + " u.college_name, u.department, u.study_year "
                    + "FROM " + REGISTRATIONS + " r "
                    + "JOIN " + EVENTS + " e ON e.id = r.event_id "
                    + "JOIN " + USERS + " u ON u.roll_no = r.participant_id "
                    + "WHERE 1 = 1");
    List<Object> parameters = new ArrayList<>();

    appendReportFilters(
            sql,
            parameters,
            eventId,
            collegeName,
            department,
            year,
            search);
    sql.append(" ORDER BY r.registered_at DESC");

    List<RegistrationRecord> records = new ArrayList<>();
```

```
        try (Connection connection = Dao.getConnection();
                PreparedStatement statement = connection.prepareStatement(sql.toString())) {

            for (int index = 0; index < parameters.size(); index++) {
                statement.setObject(index + 1, parameters.get(index));
            }

            try (ResultSet result = statement.executeQuery()) {
                while (result.next()) {
                    records.add(mapRegistrationRecord(result));
                }
            }
        }
        return records;
    }

    private void appendReportFilters(
            StringBuilder sql,
            List<Object> parameters,
            Integer eventId,
            String collegeName,
            String department,
            Integer year,
            String search) {

        if (eventId != null) {
            sql.append(" AND e.id = ?");
            parameters.add(eventId);
        }
        if (collegeName != null && !collegeName.isBlank()) {
            sql.append(" AND u.college_name = ?");
            parameters.add(collegeName);
        }
        if (department != null && !department.isBlank()) {
            sql.append(" AND u.department = ?");
            parameters.add(department);
        }
        if (year != null) {
            sql.append(" AND u.study_year = ?");
            parameters.add(year);
        }
        if (search != null && !search.isBlank()) {
            sql.append(" AND (u.roll_no LIKE ? OR u.full_name LIKE ? OR u.email LIKE ?"
                    + " OR u.phone_number LIKE ? OR r.pass_id LIKE ?)");
            String term = "%" + search.trim() + "%";
            parameters.addAll(List.of(term, term, term, term, term));
        }
    }

    public byte[] profilePhoto(String rollNumber) throws SQLException {
        String sql = "SELECT profile_photo FROM " + USERS + " WHERE roll_no = ?";
```

```
    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        try (ResultSet result = statement.executeQuery()) {
            return result.next() ? result.getBytes("profile_photo") : null;
        }
    }
}

public String profilePhotoType(String rollNumber) throws SQLException {
    String sql = "SELECT profile_photo_type FROM " + USERS + " WHERE roll_no = ?";

    try (Connection connection = Dao.getConnection();
            PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, rollNumber);
        try (ResultSet result = statement.executeQuery()) {
            return result.next() ? result.getString("profile_photo_type") : null;
        }
    }
}

private User mapUser(ResultSet result) throws SQLException {
    return new User(
            result.getString("roll_no"),
            result.getString("full_name"),
            result.getString("email"),
            result.getString("phone_number"),
            result.getString("college_name"),
            result.getString("password_hash"),
            result.getString("role"),
            result.getString("department"),
            result.getInt("study_year"),
            result.getString("gender"),
            result.getString("address"));
}

private Event mapEvent(ResultSet result) throws SQLException {
    return new Event(
            result.getInt("id"),
            result.getString("title"),
            result.getString("description"),
            result.getString("category"),
            result.getString("department"),
            result.getString("venue"),
            result.getTimestamp("event_date").toLocalDateTime(),
            result.getTimestamp("registration_closes_at").toLocalDateTime(),
            result.getInt("capacity"),
            result.getDouble("fee"),
            result.getBytes("image_data") != null,
            result.getString("organizer_id"),
```

```
                result.getBoolean("active"),
                result.getInt("registered_count"));
    }

    private RegistrationRecord mapRegistrationRecord(ResultSet result)
            throws SQLException {

        return new RegistrationRecord(
                result.getInt("event_id"),
                result.getString("title"),
                result.getString("participant_id"),
                result.getString("full_name"),
                result.getString("email"),
                result.getString("phone_number"),
                result.getString("college_name"),
                result.getString("department"),
                result.getInt("study_year"),
                result.getString("pass_id"),
                result.getString("payment_status"),
                result.getTimestamp("registered_at").toLocalDateTime());
    }
}
```

2 Dao.java

src/main/java/com/dao/Dao.java

```
package com.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public final class Dao {

    private Dao() {
    }

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new ExceptionInInitializerError("MySQL JDBC driver is missing from WEB-
            INF/lib: " + e.getMessage());
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
                env(
                        "DB_URL",
                        "jdbc:mysql://localhost:3306/event"
                                + "?createDatabaseIfNotExist=true"
                                + "&serverTimezone=Asia/Kolkata"),
                env("DB_USER", "root"),
                env("DB_PASSWORD", "[DATABASE PASSWORD]"));
    }

    public static String env(String name, String fallback) {
        String value = System.getenv(name);
        return value == null || value.isBlank() ? fallback : value;
    }
}
```

3 DatabaseSetup.java

src/main/java/com/dao/DatabaseSetup.java

```
package com.dao;

/** Command-line database initializer for local setup and verification. */
public final class DatabaseSetup {

    private DatabaseSetup() {
    }

    public static void main(String[] args) throws Exception {
        new AppDao().initialize();
        System.out.println("Campus Event database tables and organizer seed are ready.");
    }
}
```

4 Event.java

src/main/java/com/model/Event.java

```
package com.model;

import java.time.LocalDateTime;

public record Event(
        int id,
        String title,
        String description,
        String category,
        String department,
        String venue,
        LocalDateTime date,
        LocalDateTime registrationClosesAt,
        int capacity,
        double fee,
        boolean hasImage,
        String organizerId,
        boolean active,
        int registeredCount) {
}
```

5 ParticipantEventRegistration.java

src/main/java/com/model/ParticipantEventRegistration.java

```
package com.model;

import java.time.LocalDateTime;

public record ParticipantEventRegistration(
        Event event,
        String passId,
        String paymentStatus,
        LocalDateTime registeredAt) {
}
```

6 RegistrationRecord.java

src/main/java/com/model/RegistrationRecord.java

```
package com.model;

import java.time.LocalDateTime;

public record RegistrationRecord(
        int eventId,
        String eventTitle,
        String rollNo,
        String name,
        String email,
        String phoneNumber,
        String collegeName,
        String department,
        int year,
        String passId,
        String paymentStatus,
        LocalDateTime registeredAt) {
}
```

7 User.java

src/main/java/com/model/User.java

```
package com.model;

import java.io.Serializable;

public final class User implements Serializable {

    private static final long serialVersionUID = 1L;

    private final String rollNo;
    private final String fullName;
    private final String email;
    private final String phoneNumber;
    private final String collegeName;
    private final String passwordHash;
    private final String role;
    private final String department;
    private final int year;
    private final String gender;
    private final String address;

    public User(
            String rollNo,
            String fullName,
            String email,
            String phoneNumber,
            String collegeName,
            String passwordHash,
            String role,
            String department,
            int year,
            String gender,
            String address) {

        this.rollNo = rollNo;
        this.fullName = fullName;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.collegeName = collegeName;
        this.passwordHash = passwordHash;
        this.role = role;
        this.department = department;
        this.year = year;
        this.gender = gender;
        this.address = address;
    }

    public String rollNo() {
        return rollNo;
```

```
    }

    public String fullName() {
        return fullName;
    }

    public String email() {
        return email;
    }

    public String phoneNumber() {
        return phoneNumber;
    }

    public String collegeName() {
        return collegeName;
    }

    public String passwordHash() {
        return passwordHash;
    }

    public String role() {
        return role;
    }

    public String department() {
        return department;
    }

    public int year() {
        return year;
    }

    public String gender() {
        return gender;
    }

    public String address() {
        return address;
    }
}
```

8 EmailService.java

src/main/java/com/service/EmailService.java

```
package com.service;

import com.dao.Dao;
import com.model.Event;
import com.model.User;

import java.io.UnsupportedEncodingException;
import java.nio.charset.StandardCharsets;
import java.time.format.DateTimeFormatter;
import java.util.Properties;

import javax.mail.Authenticator;
import javax.mail.Message;
import javax.mail.MessagingException;
import javax.mail.PasswordAuthentication;
import javax.mail.Session;
import javax.mail.Transport;
import javax.mail.internet.InternetAddress;
import javax.mail.internet.MimeMessage;

public final class EmailService {

    private static final String SMTP_APP_PASSWORD = "[SMTP APP PASSWORD]";
    private static final DateTimeFormatter EVENT_DATE_FORMAT =
            DateTimeFormatter.ofPattern("dd MMMM yyyy, hh:mm a");

    private EmailService() {
    }

    public static void sendRegistrationOtp(String email, String otp)
            throws MessagingException {

        String content = paragraph(
                "We received a request to verify this email address for a new "
                        + "Campus Events participant account.")
                + codePanel("Verification code", otp)
                + paragraph("This code is valid for 5 minutes. Do not share it with anyone.")
                + securityNotice(
                        "If you did not request this registration, you may safely ignore this email.");

        sendHtml(
                email,
                "Verify your Campus Events email address",
                "Participant email verification",
                "Complete your email verification",
                content);
    }
```

```
    public static void sendOrganizerLoginOtp(User organizer, String otp)
            throws MessagingException {

        String content = paragraph(
                "Hello " + escapeHtml(organizer.fullName())
                        + ", use the secure code below to complete your organizer login.")
                + codePanel("Organizer login code", otp)
                + paragraph("This code expires in 5 minutes and can be used only once.")
                + securityNotice(
                        "If you did not attempt to sign in, please change your password and "
                                + "contact the system administrator.");

        sendHtml(
                organizer.email(),
                "Your Campus Events organizer login code",
                "Secure organizer sign-in",
                "Organizer login verification",
                content);
    }

    public static void sendParticipantWelcome(User participant)
            throws MessagingException {

        String content = paragraph(
                "Hello " + escapeHtml(participant.fullName())
                        + ", your participant account has been created successfully.")
                + detailTable(
                        new String[][] {
                            {"Roll number / Username", participant.rollNo()},
                            {"Registered email", participant.email()},
                            {"Department", participant.department()},
                            {"Study year", String.valueOf(participant.year())}
                        })
                + paragraph(
                        "You can now sign in with your roll number and password to browse "
                                + "available campus events.");

        sendHtml(
                participant.email(),
                "Welcome to Campus Events",
                "Participant registration completed",
                "Registration successful",
                content);
    }

    public static void sendEventRegistrationReceipt(
            User participant,
            Event event,
            String passId) throws MessagingException {

        String content = paragraph(
                "Hello " + escapeHtml(participant.fullName())
```

```
                + ", your event registration and payment have been confirmed.")
            + detailTable(
                new String[][] {
                    {"Event", event.title()},
                    {"Date and time", event.date().format(EVENT_DATE_FORMAT)},
                    {"Venue", event.venue()},
                    {"Pass ID", passId},
                    {"Payment status", "PAID"},
                    {"Amount", String.format("₹%.2f", event.fee())}
                })
            + securityNotice(
                "Please keep this email or your Pass ID available when entering the event.");

        sendHtml(
            participant.email(),
            "Event registration confirmed — " + event.title(),
            "Event registration receipt",
            "Your event registration is confirmed",
            content);
    }

    private static void sendHtml(
            String recipient,
            String subject,
            String eyebrow,
            String heading,
            String content) throws MessagingException {

        String smtpHost = Dao.env("SMTP_HOST", "smtp.gmail.com");
        String smtpUser = Dao.env("SMTP_USER", "nitwg6@gmail.com");

        Properties properties = new Properties();
        properties.put("mail.smtp.host", smtpHost);
        properties.put("mail.smtp.port", Dao.env("SMTP_PORT", "587"));
        properties.put("mail.smtp.auth", "true");
        properties.put("mail.smtp.starttls.enable", "true");
        properties.put("mail.smtp.starttls.required", "true");

        Session mailSession = Session.getInstance(properties, new Authenticator() {
            @Override
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication(smtpUser, SMTP_APP_PASSWORD);
            }
        });

        MimeMessage message = new MimeMessage(mailSession);
        InternetAddress sender = new InternetAddress(Dao.env("SMTP_FROM", smtpUser));

        try {
            sender.setPersonal("Campus Events", StandardCharsets.UTF_8.name());
        } catch (UnsupportedEncodingException exception) {
            throw new MessagingException("Unable to create the sender address", exception);
```

```
        }

        message.setFrom(sender);
        message.setRecipients(
                Message.RecipientType.TO,
                InternetAddress.parse(recipient));
        message.setSubject(subject, StandardCharsets.UTF_8.name());
        message.setContent(emailTemplate(eyebrow, heading, content), "text/html; charset=UTF-8");

        Transport.send(message);
    }

    private static String emailTemplate(String eyebrow, String heading, String content) {
        return "<!doctype html><html><body style=\"margin:0;background:#f3f6f8;"
                + "font-family:Arial,sans-serif;color:#17212b\">"
                + "<div style=\"max-width:620px;margin:0 auto;padding:28px 16px\">"
                + "<div style=\"background:#12355b;color:#fff;padding:20px 24px;"
                + "border-radius:8px 8px 0 0\">"
                + "<div style=\"font-size:12px;letter-spacing:1px;text-transform:uppercase;"
                + "opacity:.85\">" + escapeHtml(eyebrow) + "</div>"
                + "<div style=\"font-size:24px;font-weight:700;margin-top:6px\">"
                + escapeHtml(heading) + "</div></div>"
                + "<div style=\"background:#fff;border:1px solid #d8e1e8;border-top:0;"
                + "padding:26px 24px;border-radius:0 0 8px 8px\">"
                + content
                + "<p style=\"margin:26px 0 0;color:#52616f;font-size:13px;"
                + "border-top:1px solid #d8e1e8;padding-top:18px\">"
                + "Campus Event Management System<br>Automated message — please do not reply."
                + "</p></div></div></body></html>";
    }

    private static String paragraph(String text) {
        return "<p style=\"font-size:15px;line-height:1.6;margin:0 0 18px\">"
                + text + "</p>";
    }

    private static String codePanel(String label, String code) {
        return "<div style=\"background:#eef4f8;border:1px solid #d8e1e8;"
                + "border-radius:6px;padding:18px;text-align:center;margin:18px 0\">"
                + "<div style=\"font-size:12px;color:#52616f;text-transform:uppercase;"
                + "letter-spacing:.8px\">" + escapeHtml(label) + "</div>"
                + "<div style=\"font-size:30px;font-weight:700;letter-spacing:6px;"
                + "color:#12355b;margin-top:8px\">" + escapeHtml(code) + "</div></div>";
    }

    private static String detailTable(String[][] rows) {
        StringBuilder table = new StringBuilder(
                "<table role=\"presentation\" style=\"width:100%;border-collapse:collapse;"
                        + "margin:18px 0\">");

        for (String[] row : rows) {
            table.append("<tr><td style=\"padding:10px;border-bottom:1px solid #d8e1e8;"
```

```
                + "color:#52616f;width:42%\">")
                .append(escapeHtml(row[0]))
                .append("</td><td style=\"padding:10px;border-bottom:1px solid #d8e1e8;"
                        + "font-weight:600\">")
                .append(escapeHtml(row[1]))
                .append("</td></tr>");
        }

        return table.append("</table>").toString();
    }

    private static String securityNotice(String text) {
        return "<div style=\"background:#fff8e6;border-left:4px solid #9a7116;"
                + "padding:12px 14px;font-size:13px;line-height:1.5\">"
                + text + "</div>";
    }

    private static String escapeHtml(String value) {
        if (value == null) {
            return "";
        }

        return value.replace("&", "&amp;")
                .replace("<", "&lt;")
                .replace(">", "&gt;")
                .replace("\"", "&quot;")
                .replace("'", "&#39;");
    }
}
```

9 AppContextListener.java

src/main/java/com/servlets/AppContextListener.java

```
package com.servlets;

import com.dao.AppDao;
import com.mysql.cj.jdbc.AbandonedConnectionCleanupThread;

import java.sql.Driver;
import java.sql.DriverManager;
import java.util.Enumeration;

import javax.servlet.ServletContextEvent;
import javax.servlet.ServletContextListener;
import javax.servlet.annotation.WebListener;

@WebListener
public class AppContextListener implements ServletContextListener {

    @Override
    public void contextInitialized(ServletContextEvent event) {
        try {
            new AppDao().initialize();
        } catch (Exception exception) {
            event.getServletContext().log("Database initialization failed", exception);
        }
    }

    @Override
    public void contextDestroyed(ServletContextEvent event) {
        try {
            AbandonedConnectionCleanupThread.checkedShutdown();
        } catch (Exception exception) {
            event.getServletContext().log(
                    "Unable to stop the MySQL cleanup thread",
                    exception);
        }

        ClassLoader applicationClassLoader = getClass().getClassLoader();
        Enumeration<Driver> drivers = DriverManager.getDrivers();

        while (drivers.hasMoreElements()) {
            Driver driver = drivers.nextElement();
            if (driver.getClass().getClassLoader() != applicationClassLoader) {
                continue;
```

```
            }

            try {
                DriverManager.deregisterDriver(driver);
            } catch (Exception exception) {
                event.getServletContext().log(
                        "Unable to unregister JDBC driver " + driver,
                        exception);
            }
        }
    }
}
```

10 AuthFilter.java

src/main/java/com/servlets/AuthFilter.java

```
package com.servlets;

import java.io.IOException;

import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebFilter(urlPatterns = {
    "/dashboard",
    "/events",
    "/registrations",
    "/profile",
    "/event",
    "/registration",
    "/reports",
    "/event-image",
    "/profile-photo",
    "/logout"
})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest servletRequest,
            ServletResponse servletResponse,
            FilterChain chain) throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) servletRequest;
        HttpServletResponse response = (HttpServletResponse) servletResponse;
        HttpSession session = request.getSession(false);

        if (session == null || session.getAttribute("user") == null) {
```

```
            response.sendRedirect(
                    request.getContextPath() + "/login?status=session-required");
            return;
        }

        response.setHeader("Cache-Control", "no-store, no-cache, must-revalidate");
        response.setHeader("Pragma", "no-cache");
        response.setDateHeader("Expires", 0);

        chain.doFilter(request, response);
    }
}
```

11 DashboardServlet.java

src/main/java/com/servlets/DashboardServlet.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.Event;
import com.model.ParticipantEventRegistration;
import com.model.User;

import java.io.IOException;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet(urlPatterns = {
    "/dashboard",
    "/events",
    "/registrations",
    "/profile"
})
public class DashboardServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        try {
            AppDao appDao = new AppDao();
            User user = (User) request.getSession().getAttribute("user");
            request.setAttribute("activeSection", activeSection(request));

            if ("PARTICIPANT".equals(user.role())) {
                List<Event> availableEvents = appDao.events(true);
                List<ParticipantEventRegistration> registrations =
                        appDao.participantRegistrations(user.rollNo());

                if ("events".equals(request.getAttribute("activeSection"))) {
                    availableEvents = filterEvents(
                            availableEvents,
                            request.getParameter("category"),
                            request.getParameter("department"));
                } else if ("registrations".equals(
                        request.getAttribute("activeSection"))) {
                    registrations = filterRegistrations(
```

```
                    registrations,
                    request.getParameter("category"),
                    request.getParameter("department"));
            }

            request.setAttribute("availableEvents", availableEvents);
            request.setAttribute(
                    "participantRegistrations",
                    registrations);
            request.getRequestDispatcher("/WEB-INF/views/userdashboard.jsp")
                    .forward(request, response);
            return;
        }

        if ("registrations".equals(request.getAttribute("activeSection"))) {
            response.sendRedirect(request.getContextPath() + "/reports");
            return;
        }

        List<Event> events = appDao.events(false);
        if ("events".equals(request.getAttribute("activeSection"))) {
            events = filterEvents(
                    events,
                    request.getParameter("category"),
                    request.getParameter("department"));
        }
        request.setAttribute("events", events);
        request.getRequestDispatcher("/WEB-INF/views/organizerdashboard.jsp")
                .forward(request, response);
    } catch (Exception exception) {
        throw new ServletException("Unable to load dashboard.", exception);
    }
}

private String activeSection(HttpServletRequest request) {
    return switch (request.getServletPath()) {
        case "/events" -> "events";
        case "/registrations" -> "registrations";
        case "/profile" -> "profile";
        default -> "dashboard";
    };
}

private List<Event> filterEvents(
        List<Event> events,
        String category,
        String department) {

    return events.stream()
            .filter(event -> category == null
                    || category.isBlank()
                    || category.equals(event.category()))
```

```
                .filter(event -> department == null
                        || department.isBlank()
                        || department.equals(event.department()))
                .toList();
    }

    private List<ParticipantEventRegistration> filterRegistrations(
            List<ParticipantEventRegistration> registrations,
            String category,
            String department) {

        return registrations.stream()
                .filter(registration -> category == null
                        || category.isBlank()
                        || category.equals(registration.event().category()))
                .filter(registration -> department == null
                        || department.isBlank()
                        || department.equals(registration.event().department()))
                .toList();
    }
}
```

12 EventImageServlet.java

src/main/java/com/servlets/EventImageServlet.java

```
package com.servlets;

import com.dao.AppDao;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/event-image")
public class EventImageServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        try {
            int eventId = Integer.parseInt(request.getParameter("id"));
            AppDao appDao = new AppDao();
            byte[] image = appDao.eventImage(eventId);

            if (image == null || image.length == 0) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND);
                return;
            }

            String imageType = appDao.eventImageType(eventId);
            response.setContentType(imageType == null ? "image/jpeg" : imageType);
            response.setContentLength(image.length);
            response.setHeader("Cache-Control", "private, max-age=3600");
            response.getOutputStream().write(image);
        } catch (NumberFormatException exception) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid event ID.");
        } catch (Exception exception) {
            throw new ServletException("Unable to load event image.", exception);
        }
    }
}
```

13 EventServlet.java

src/main/java/com/servlets/EventServlet.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.Event;
import com.model.User;

import java.io.IOException;
import java.io.InputStream;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.Set;

import javax.servlet.ServletException;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.Part;

@WebServlet("/event")
@MultipartConfig(maxFileSize = 5 * 1024 * 1024)
public class EventServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private static final Set<String> ALLOWED_IMAGE_TYPES = Set.of(
            "image/jpeg",
            "image/png",
            "image/webp");
    private static final Set<String> CATEGORIES = Set.of(
            "Technical", "Cultural", "Sports", "Workshop", "Seminar");
    private static final Set<String> DEPARTMENTS = Set.of(
            "CSE", "IT", "AI&DS", "CSM", "EEE", "ECE", "MECH", "CIVIL", "ASE");

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String eventId = request.getParameter("id");
        User user = (User) request.getSession().getAttribute("user");

        if (eventId == null || eventId.isBlank()) {
            if (!"ORGANIZER".equals(user.role())) {
                response.sendError(HttpServletResponse.SC_FORBIDDEN);
                return;
            }
```

```
            request.getRequestDispatcher("/WEB-INF/views/event-form.jsp")
                    .forward(request, response);
            return;
        }

        try {
            Event event = new AppDao().event(Integer.parseInt(eventId));
            if (event == null) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND, "Event not found.");
                return;
            }

            request.setAttribute("event", event);
            request.setAttribute("backPath", participantBackPath(request));
            request.getRequestDispatcher("/WEB-INF/views/event-form.jsp")
                    .forward(request, response);
        } catch (NumberFormatException exception) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid event ID.");
        } catch (Exception exception) {
            throw new ServletException("Unable to load event.", exception);
        }
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        try {
            requireOrganizer(request);
            AppDao appDao = new AppDao();

            if ("delete".equals(request.getParameter("action"))) {
                appDao.deleteEvent(Integer.parseInt(request.getParameter("id")));
            } else {
                User organizer = (User) request.getSession().getAttribute("user");
                String eventId = request.getParameter("id");
                int id = eventId == null || eventId.isBlank()
                        ? 0
                        : Integer.parseInt(eventId);
                LocalDateTime eventDateTime = dateTime(
                        required(request, "eventDate", "Event date"),
                        required(request, "eventTime", "Event start time"));
                LocalDateTime registrationClosesAt = dateTime(
                        required(
                                request,
                                "registrationCloseDate",
                                "Registration closing date"),
                        required(
                                request,
                                "registrationCloseTime",
                                "Registration closing time"));
```

```
        validateDates(eventDateTime, registrationClosesAt);

        Part imagePart = request.getPart("eventImage");
        byte[] imageData = readImage(imagePart);
        if (id == 0 && imageData == null) {
            throw new IllegalArgumentException("Event image is required.");
        }
        String imageType = imageData == null ? null : imagePart.getContentType();
        boolean hasImage = imageData != null;

        if (!hasImage && id > 0) {
            Event existingEvent = appDao.event(id);
            hasImage = existingEvent != null && existingEvent.hasImage();
        }

        int capacity = Integer.parseInt(required(
                request,
                "capacity",
                "Capacity"));
        double fee = Double.parseDouble(required(request, "fee", "Fee"));
        String category = required(request, "category", "Category");
        String department = required(request, "department", "Department");

        if (capacity < 1 || fee < 0) {
            throw new IllegalArgumentException(
                    "Capacity must be at least 1 and fee cannot be negative.");
        }
        if (!CATEGORIES.contains(category)) {
            throw new IllegalArgumentException("Select a valid event category.");
        }
        if (!DEPARTMENTS.contains(department)) {
            throw new IllegalArgumentException("Select a valid department.");
        }

        Event event = new Event(
                id,
                required(request, "title", "Event title"),
                required(request, "description", "Description"),
                category,
                department,
                required(request, "venue", "Venue"),
                eventDateTime,
                registrationClosesAt,
                capacity,
                fee,
                hasImage,
                organizer.rollNo(),
                request.getParameter("active") != null,
                0);

        appDao.saveEvent(event, imageData, imageType);
    }
```

```
            response.sendRedirect(request.getContextPath() + "/events");
        } catch (SecurityException exception) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN);
        } catch (NumberFormatException exception) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid event data.");
        } catch (IllegalArgumentException exception) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, exception.getMessage());
        } catch (Exception exception) {
            throw new ServletException("Unable to save event.", exception);
        }
    }

    private String participantBackPath(HttpServletRequest request) {
        String source = request.getParameter("from");
        return "registrations".equals(source) ? "registrations" : "events";
    }

    private String required(
            HttpServletRequest request,
            String parameter,
            String label) {

        String value = request.getParameter(parameter);
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(label + " is required.");
        }
        return value.trim();
    }

    private LocalDateTime dateTime(String date, String time) {
        return LocalDateTime.of(LocalDate.parse(date), LocalTime.parse(time));
    }

    private void validateDates(
            LocalDateTime eventDateTime,
            LocalDateTime registrationClosesAt) {

        if (!registrationClosesAt.isBefore(eventDateTime)) {
            throw new IllegalArgumentException(
                    "Registration closing time must be before the event starts.");
        }

        if (registrationClosesAt.isBefore(LocalDateTime.now())) {
            throw new IllegalArgumentException(
                    "Registration closing date and time must be in the future.");
        }
    }

    private byte[] readImage(Part imagePart) throws IOException, ServletException {
        if (imagePart == null || imagePart.getSize() == 0) {
            return null;
```

```
        }
        if (!ALLOWED_IMAGE_TYPES.contains(imagePart.getContentType())) {
            throw new IllegalArgumentException("Event image must be JPG, PNG, or WEBP.");
        }

        try (InputStream input = imagePart.getInputStream()) {
            return input.readAllBytes();
        }
    }

    private void requireOrganizer(HttpServletRequest request) {
        User user = (User) request.getSession().getAttribute("user");
        if (!"ORGANIZER".equals(user.role())) {
            throw new SecurityException("Organizer access required.");
        }
    }
}
```

14 Login.java

src/main/java/com/servlets/Login.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.User;
import com.service.EmailService;
import com.util.PasswordUtil;

import java.io.IOException;
import java.security.SecureRandom;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/login")
public class Login extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private static final int AUTHENTICATED_SESSION_SECONDS = 30 * 60;
    private static final int OTP_SESSION_SECONDS = 5 * 60;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null && session.getAttribute("user") != null) {
            response.sendRedirect(request.getContextPath() + "/dashboard");
            return;
        }

        String status = request.getParameter("status");
        if ("registered".equals(status)) {
            request.setAttribute(
                    "statusMessage",
                    "Registration completed successfully. Please sign in.");
        } else if ("logged-out".equals(status)) {
            request.setAttribute("statusMessage", "You have been logged out securely.");
        } else if ("session-required".equals(status)) {
            request.setAttribute("statusMessage", "Please sign in to continue.");
        }

        request.getRequestDispatcher("/WEB-INF/views/login.jsp")
                .forward(request, response);
    }
```

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    String submittedRollNumber = request.getParameter("rollNo");
    String password = request.getParameter("password");

    if (submittedRollNumber == null || submittedRollNumber.isBlank()
            || password == null || password.isBlank()) {
        showLoginError(request, response, "Enter your roll number and password.");
        return;
    }

    String rollNumber = submittedRollNumber.trim().toUpperCase();

    try {
        User user = new AppDao().findUser(rollNumber);
        if (user == null || !PasswordUtil.verify(password, user.passwordHash())) {
            showLoginError(request, response, "Invalid roll number or password.");
            return;
        }

        if ("ORGANIZER".equals(user.role())) {
            beginOrganizerOtpLogin(request, response, user);
            return;
        }

        createAuthenticatedSession(request, user);
        response.sendRedirect(request.getContextPath() + "/dashboard");
    } catch (Exception exception) {
        getServletContext().log("Login failed", exception);
        showLoginError(
                request,
                response,
                "Login is temporarily unavailable. Please try again shortly.");
    }
}

private void beginOrganizerOtpLogin(
        HttpServletRequest request,
        HttpServletResponse response,
        User organizer) throws Exception {

    String otp = String.format(
            "%06d",
            new SecureRandom().nextInt(1_000_000));

    HttpSession pendingSession = replaceSession(request);
    pendingSession.setAttribute("loginOtp", otp);
    pendingSession.setAttribute("pendingUser", organizer);
    pendingSession.setAttribute(
```

```
                "otpExpiresAt",
                System.currentTimeMillis() + OTP_SESSION_SECONDS * 1_000L);
        pendingSession.setMaxInactiveInterval(OTP_SESSION_SECONDS);

        EmailService.sendOrganizerLoginOtp(organizer, otp);
        response.sendRedirect(request.getContextPath() + "/verify-otp");
    }

    private void createAuthenticatedSession(HttpServletRequest request, User user) {
        HttpSession authenticatedSession = replaceSession(request);
        authenticatedSession.setAttribute("user", user);
        authenticatedSession.setMaxInactiveInterval(AUTHENTICATED_SESSION_SECONDS);
    }

    private HttpSession replaceSession(HttpServletRequest request) {
        HttpSession existingSession = request.getSession(false);
        if (existingSession != null) {
            existingSession.invalidate();
        }
        return request.getSession(true);
    }

    private void showLoginError(
            HttpServletRequest request,
            HttpServletResponse response,
            String message) throws ServletException, IOException {

        request.setAttribute("errorMessage", message);
        request.getRequestDispatcher("/WEB-INF/views/login.jsp")
                .forward(request, response);
    }
}
```

15 LogoutServlet.java

src/main/java/com/servlets/LogoutServlet.java

```
package com.servlets;

import java.io.IOException;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect(request.getContextPath() + "/login?status=logged-out");
    }
}
```

16 OtpServlet.java

src/main/java/com/servlets/OtpServlet.java

```
package com.servlets;

import com.model.User;

import java.io.IOException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/verify-otp")
public class OtpServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private static final int AUTHENTICATED_SESSION_SECONDS = 30 * 60;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session == null || session.getAttribute("pendingUser") == null) {
            response.sendRedirect(request.getContextPath() + "/login?status=session-required");
            return;
        }

        request.getRequestDispatcher("/WEB-INF/views/verify-otp.jsp")
                .forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        String submittedOtp = request.getParameter("otp");
        String expectedOtp = session == null
                ? null
                : (String) session.getAttribute("loginOtp");
        Long expiresAt = session == null
                ? null
                : (Long) session.getAttribute("otpExpiresAt");

        boolean valid = expectedOtp != null
                && expiresAt != null
```

```
                && System.currentTimeMillis() <= expiresAt
                && expectedOtp.equals(submittedOtp);

        if (!valid) {
            request.setAttribute("errorMessage", "The OTP is invalid or has expired.");
            request.getRequestDispatcher("/WEB-INF/views/verify-otp.jsp")
                    .forward(request, response);
            return;
        }

        User organizer = (User) session.getAttribute("pendingUser");
        session.invalidate();

        HttpSession authenticatedSession = request.getSession(true);
        authenticatedSession.setAttribute("user", organizer);
        authenticatedSession.setMaxInactiveInterval(AUTHENTICATED_SESSION_SECONDS);

        response.sendRedirect(request.getContextPath() + "/dashboard");
    }
}
```

17 ProfilePhotoServlet.java

src/main/java/com/servlets/ProfilePhotoServlet.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.User;

import java.io.IOException;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/profile-photo")
public class ProfilePhotoServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        User viewer = (User) request.getSession().getAttribute("user");
        String requestedRollNumber = request.getParameter("rollNo");
        String rollNumber = requestedRollNumber == null || requestedRollNumber.isBlank()
                ? viewer.rollNo()
                : requestedRollNumber.trim().toUpperCase();

        boolean canView = "ORGANIZER".equals(viewer.role())
                || viewer.rollNo().equals(rollNumber);
        if (!canView) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN);
            return;
        }

        try {
            AppDao appDao = new AppDao();
            byte[] profilePhoto = appDao.profilePhoto(rollNumber);
            if (profilePhoto == null || profilePhoto.length == 0) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND);
                return;
            }
```

```
            String contentType = appDao.profilePhotoType(rollNumber);
            response.setContentType(contentType == null ? "image/jpeg" : contentType);
            response.setContentLength(profilePhoto.length);
            response.setHeader("Cache-Control", "private, max-age=300");
            response.getOutputStream().write(profilePhoto);
        } catch (Exception exception) {
            getServletContext().log("Profile photo load failed", exception);
            response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
        }
    }
}
```

18 RegisterServlet.java

src/main/java/com/servlets/RegisterServlet.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.User;
import com.service.EmailService;
import com.util.PasswordUtil;

import java.io.IOException;
import java.util.Set;

import javax.servlet.ServletException;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import javax.servlet.http.Part;

@WebServlet("/register")
@MultipartConfig(maxFileSize = 2_097_152)
public class RegisterServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private static final Set<String> DEPARTMENTS = Set.of(
            "CSE", "IT", "AI&DS", "CSM", "EEE", "ECE", "MECH", "CIVIL", "ASE");
    private static final Set<String> COLLEGES = Set.of(
            "Lakireddy Bali Reddy college of Engineering",
            "Bapatla Engineering college",
            "SRK Institute of Technology",
            "Andhra Loyola Institute of Engineering and Technology",
            "Velagapudi Ramakrishna Siddhartha Engineering College",
            "PSCMR College of Engineering and Technology",
            "Prasad V. Potluri Siddhartha Institute Of Technology",
            "RK COLLEGE OF ENGINEERING",
            "MIC College of Technology",
            "Amrita Sai Institute of Science and Technology");
    private static final Set<String> GENDERS = Set.of("Male", "Female", "Other");

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null && session.getAttribute("user") != null) {
            response.sendRedirect(request.getContextPath() + "/dashboard");
            return;
        }
```

```
        request.getRequestDispatcher("/WEB-INF/views/register.jsp")
                .forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String rollNumber = normalize(request.getParameter("rollNo")).toUpperCase();
        String email = normalize(request.getParameter("email")).toLowerCase();
        String password = normalize(request.getParameter("password"));

        HttpSession session = request.getSession(false);
        String verifiedEmail = session == null
                ? null
                : (String) session.getAttribute("verifiedRegistrationEmail");

        if (verifiedEmail == null || !verifiedEmail.equals(email)) {
            request.setAttribute(
                    "errorMessage",
                    "Verify this email address with OTP before submitting.");
            request.getRequestDispatcher("/WEB-INF/views/register.jsp")
                    .forward(request, response);
            return;
        }

        AppDao appDao = new AppDao();
        boolean participantCreated = false;

        try {
            validateRegistration(request, rollNumber, password);

            Part profilePhotoPart = request.getPart("profilePhoto");
            validateProfilePhoto(profilePhotoPart);

            String conflict = appDao.registrationConflict(
                    rollNumber,
                    email,
                    normalize(request.getParameter("phoneNumber")));
            if (conflict != null) {
                throw new IllegalArgumentException(conflict);
            }

            byte[] profilePhoto = profilePhotoPart.getInputStream().readAllBytes();
            User participant = buildParticipant(request, rollNumber, email);

            appDao.createParticipant(
                    participant,
                    PasswordUtil.hash(password),
                    profilePhoto,
                    profilePhotoPart.getContentType());
```

```
            participantCreated = true;
            session.removeAttribute("verifiedRegistrationEmail");

            sendRegistrationConfirmation(participant);

            response.sendRedirect(
                    request.getContextPath() + "/login?status=registered");
        } catch (Exception exception) {
            if (participantCreated) {
                removeIncompleteParticipant(appDao, rollNumber);
            }

            getServletContext().log("Participant registration failed", exception);
            request.setAttribute("errorMessage", registrationErrorMessage(exception));
            request.getRequestDispatcher("/WEB-INF/views/register.jsp")
                    .forward(request, response);
        }
    }

    private void validateRegistration(
            HttpServletRequest request,
            String rollNumber,
            String password) {

        String confirmPassword = requireText(
                request,
                "confirmPassword",
                "Confirm password");

        requireText(request, "fullName", "Full name");
        requireText(request, "rollNo", "Roll number");
        requireText(request, "email", "Email");
        requireText(request, "phoneNumber", "Phone number");
        requireText(request, "collegeName", "College name");
        requireText(request, "department", "Department");
        requireText(request, "year", "Year");
        requireText(request, "gender", "Gender");
        requireText(request, "address", "Address");

        if (!password.equals(confirmPassword)) {
            throw new IllegalArgumentException("Passwords do not match.");
        }

        if (!rollNumber.matches("[A-Z0-9]{5,20}")) {
            throw new IllegalArgumentException(
                    "Roll number must contain 5 to 20 letters or numbers.");
        }

        String phoneNumber = normalize(request.getParameter("phoneNumber"));
        if (!phoneNumber.matches("[6-9][0-9]{9}")) {
            throw new IllegalArgumentException("Enter a valid 10-digit mobile number.");
```

```
        }

        String collegeName = normalize(request.getParameter("collegeName"));
        if (!COLLEGES.contains(collegeName)) {
            throw new IllegalArgumentException("Select a valid college name.");
        }

        String department = normalize(request.getParameter("department"));
        if (!DEPARTMENTS.contains(department)) {
            throw new IllegalArgumentException("Select a valid department.");
        }

        String gender = normalize(request.getParameter("gender"));
        if (!GENDERS.contains(gender)) {
            throw new IllegalArgumentException("Select a valid gender.");
        }

        int year;
        try {
            year = Integer.parseInt(normalize(request.getParameter("year")));
        } catch (NumberFormatException exception) {
            throw new IllegalArgumentException("Select a valid study year.");
        }
        if (year < 1 || year > 4) {
            throw new IllegalArgumentException("Select a valid study year.");
        }

        String strongPasswordPattern =
                "^(?=.*[A-Z])(?=.*[a-z])(?=.*\\d)"
                        + "(?=.*[^A-Za-z0-9]).{6,15}$";

        if (!password.matches(strongPasswordPattern)) {
            throw new IllegalArgumentException(
                    "Password must be 6 to 15 characters and contain at least one "
                            + "lowercase letter, uppercase letter, number and special symbol.");
        }
    }

    private String requireText(
            HttpServletRequest request,
            String parameter,
            String label) {

        String value = request.getParameter(parameter);
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(label + " is required.");
        }
        return value.trim();
    }

    private String normalize(String value) {
        return value == null ? "" : value.trim();
```

```
    }

    private void validateProfilePhoto(Part profilePhotoPart) {
        if (profilePhotoPart == null || profilePhotoPart.getSize() == 0) {
            throw new IllegalArgumentException("Profile photo is required.");
        }

        String contentType = profilePhotoPart.getContentType();
        boolean supportedType = contentType != null
                && contentType.matches("image/(jpeg|png|webp)");

        if (!supportedType) {
            throw new IllegalArgumentException(
                    "Upload a valid JPG, PNG or WEBP profile photo.");
        }
    }

    private User buildParticipant(
            HttpServletRequest request,
            String rollNumber,
            String email) {

        return new User(
                rollNumber,
                request.getParameter("fullName").trim(),
                email,
                request.getParameter("phoneNumber").trim(),
                request.getParameter("collegeName").trim(),
                null,
                "PARTICIPANT",
                request.getParameter("department").trim(),
                Integer.parseInt(request.getParameter("year")),
                request.getParameter("gender").trim(),
                request.getParameter("address").trim());
    }

    private void sendRegistrationConfirmation(User participant) {
        try {
            EmailService.sendParticipantWelcome(participant);
        } catch (Exception mailException) {
            getServletContext().log(
                    "Registration saved but confirmation email failed",
                    mailException);
        }
    }

    private void removeIncompleteParticipant(AppDao appDao, String rollNumber) {
        try {
            appDao.deleteUser(rollNumber);
        } catch (Exception cleanupException) {
            getServletContext().log(
                    "Unable to remove incomplete participant",
```

```
                cleanupException);
        }
    }

    private String registrationErrorMessage(Exception exception) {
        String message = exception.getMessage();

        if (message != null && message.toLowerCase().contains("duplicate")) {
            String lowerMessage = message.toLowerCase();
            if (lowerMessage.contains("phone")) {
                return "Phone number already exists.";
            }
            if (lowerMessage.contains("email")) {
                return "Email address already exists.";
            }
            if (lowerMessage.contains("primary") || lowerMessage.contains("roll")) {
                return "Roll number already exists.";
            }
            return "Roll number, email address, or phone number already exists.";
        }

        return message == null ? "Registration failed." : message;
    }
}
```

19 RegistrationEmailOtpServlet.java

src/main/java/com/servlets/RegistrationEmailOtpServlet.java

```
package com.servlets;

import com.service.EmailService;

import java.io.IOException;
import java.security.SecureRandom;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/registration-email-otp")
public class RegistrationEmailOtpServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private static final long OTP_VALIDITY_MILLIS = 5 * 60 * 1_000L;

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {

        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");

        String action = request.getParameter("action");
        String email = request.getParameter("email");
        if (email == null || !email.matches("^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$")) {
            writeResponse(response, false, "Enter a valid email address.");
            return;
        }

        HttpSession session = request.getSession(true);
        try {
            if ("send".equals(action)) {
                sendOtp(session, email);
                writeResponse(response, true, "OTP sent successfully. Please check your email.");
                return;
            }

            if ("verify".equals(action) && isValidOtp(request, session, email)) {
                session.setAttribute("verifiedRegistrationEmail", email.toLowerCase());
                clearPendingOtp(session);
                writeResponse(
                        response,
                        true,
                        "Email verified successfully. You can now submit the form.");
```

```
            return;
        }

        writeResponse(response, false, "The OTP is invalid or has expired.");
    } catch (Exception exception) {
        getServletContext().log("Registration email OTP failed", exception);
        writeResponse(
                response,
                false,
                "The OTP service is temporarily unavailable. Please try again.");
    }
}

private void sendOtp(HttpSession session, String email) throws Exception {
    String otp = String.format(
            "%06d",
            new SecureRandom().nextInt(1_000_000));

    session.setAttribute("registrationOtp", otp);
    session.setAttribute("registrationOtpEmail", email.toLowerCase());
    session.setAttribute(
            "registrationOtpExpires",
            System.currentTimeMillis() + OTP_VALIDITY_MILLIS);
    session.removeAttribute("verifiedRegistrationEmail");

    EmailService.sendRegistrationOtp(email, otp);
}

private boolean isValidOtp(
        HttpServletRequest request,
        HttpSession session,
        String email) {

    String submittedOtp = request.getParameter("otp");
    String expectedOtp = (String) session.getAttribute("registrationOtp");
    String expectedEmail = (String) session.getAttribute("registrationOtpEmail");
    Long expiresAt = (Long) session.getAttribute("registrationOtpExpires");

    return expiresAt != null
            && System.currentTimeMillis() <= expiresAt
            && expectedEmail != null
            && email.equalsIgnoreCase(expectedEmail)
            && submittedOtp != null
            && submittedOtp.equals(expectedOtp);
}

private void clearPendingOtp(HttpSession session) {
    session.removeAttribute("registrationOtp");
    session.removeAttribute("registrationOtpEmail");
    session.removeAttribute("registrationOtpExpires");
}
```

```
    private void writeResponse(
            HttpServletResponse response,
            boolean successful,
            String message) throws IOException {

        String safeMessage = message.replace("\\", "\\\\").replace("\"", "\\\"");
        response.getWriter().write(
                "{\"success\":" + successful
                        + ",\"message\":\"" + safeMessage + "\"}");
    }
}
```

20 RegistrationServlet.java

src/main/java/com/servlets/RegistrationServlet.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.Event;
import com.model.User;
import com.service.EmailService;

import java.io.IOException;
import java.sql.SQLException;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/registration")
public class RegistrationServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        User participant = (User) request.getSession().getAttribute("user");
        if (!"PARTICIPANT".equals(participant.role())) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN);
            return;
        }

        String redirectPath = "/events";
        try {
            int eventId = Integer.parseInt(request.getParameter("eventId"));
            AppDao appDao = new AppDao();
            Event event = appDao.event(eventId);

            if (event == null) {
                request.getSession().setAttribute("flash", "Event not found.");
                response.sendRedirect(request.getContextPath() + redirectPath);
                return;
            }

            String passId = appDao.register(eventId, participant.rollNo());
            sendConfirmationEmail(participant, event, passId);

            request.getSession().setAttribute(
                    "flash",
```

```
                "Registration complete. Pass ID: " + passId);
            redirectPath = "/registrations";
        } catch (NumberFormatException exception) {
            request.getSession().setAttribute("flash", "Invalid event selection.");
        } catch (SQLException exception) {
            handleRegistrationDatabaseError(request, exception);
        } catch (Exception exception) {
            getServletContext().log("Event registration failed", exception);
            request.getSession().setAttribute(
                    "flash",
                    "Registration failed. Please try again.");
        }

        response.sendRedirect(request.getContextPath() + redirectPath);
    }

    private void handleRegistrationDatabaseError(
            HttpServletRequest request,
            SQLException exception) {

        String message = exception.getMessage() == null
                ? ""
                : exception.getMessage().toLowerCase();

        if (message.contains("duplicate")) {
            request.getSession().setAttribute(
                    "flash",
                    "You are already registered for this event.");
            return;
        }

        if (message.contains("full or unavailable")) {
            request.getSession().setAttribute(
                    "flash",
                    "This event is full or no longer available for registration.");
            return;
        }

        getServletContext().log("Event registration failed", exception);
        request.getSession().setAttribute(
                "flash",
                "Registration failed. Please try again.");
```

```
    }

    private void sendConfirmationEmail(User participant, Event event, String passId) {
        try {
            EmailService.sendEventRegistrationReceipt(participant, event, passId);
        } catch (Exception exception) {
            getServletContext().log(
                    "Event registration saved but confirmation email failed",
                    exception);
        }
    }
}
```

21 ReportServlet.java

src/main/java/com/servlets/ReportServlet.java

```
package com.servlets;

import com.dao.AppDao;
import com.model.User;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/reports")
public class ReportServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        User user = (User) request.getSession().getAttribute("user");
        if (!"ORGANIZER".equals(user.role())) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN);
            return;
        }

        try {
            AppDao appDao = new AppDao();
            String eventId = request.getParameter("eventId");
            String year = request.getParameter("year");

            request.setAttribute("events", appDao.events(false));
            request.setAttribute("activeSection", "reports");
            request.setAttribute("records", appDao.report(
                    eventId == null || eventId.isBlank() ? null : Integer.valueOf(eventId),
                    request.getParameter("collegeName"),
                    request.getParameter("department"),
                    year == null || year.isBlank() ? null : Integer.valueOf(year),
                    request.getParameter("search")));
```

```
            request.getRequestDispatcher("/WEB-INF/views/organizerdashboard.jsp")
                    .forward(request, response);
            } catch (NumberFormatException exception) {
                response.sendError(
                        HttpServletResponse.SC_BAD_REQUEST,
                        "Event and year filters must contain valid numbers.");
            } catch (Exception exception) {
                throw new ServletException(exception);
            }
        }
    }
```

22 PasswordUtil.java

src/main/java/com/util/PasswordUtil.java

```
package com.util;

import org.mindrot.jbcrypt.BCrypt;

public final class PasswordUtil {

    private static final int BCRYPT_COST = 12;

    private PasswordUtil() {
    }

    public static String hash(String plainTextPassword) {
        if (plainTextPassword == null || plainTextPassword.isEmpty()) {
            throw new IllegalArgumentException("Password must not be empty.");
        }

        return BCrypt.hashpw(
                plainTextPassword,
                BCrypt.gensalt(BCRYPT_COST));
    }

    public static boolean verify(String plainTextPassword, String storedHash) {
        if (plainTextPassword == null
                || storedHash == null
                || !storedHash.startsWith("$2")) {
            return false;
        }

        try {
            return BCrypt.checkpw(plainTextPassword, storedHash);
        } catch (IllegalArgumentException exception) {
            return false;
        }
    }
}
```

RESULTS

CONCLUSION

The Campus Event Management System provides a practical full stack solution for campus event publication and participant registration. Its participant and organizer roles support email verification, authenticated sessions, event browsing, controlled registration, pass retrieval, event maintenance, and registration reports in one interface.

Unique roll numbers, email addresses, and phone numbers improve identity consistency. BCrypt cost 12 protects stored passwords. The session filter and Servlet role checks restrict access, while prepared statements and relational constraints support safe and consistent database operations. OTP email messages and registration receipts complete the main communication workflows.

The application separates browser presentation, Servlet controllers, models, data access, and email services. This structure makes the project suitable for demonstrating HTML, CSS, JavaScript, JSP, Servlets, JDBC, MySQL, and Tomcat together. Category and department filters and detailed reports help both participant navigation and organizer administration.

The current system treats payment as a classroom simulation and records PAID without a real gateway. Future work can add payment verification, password recovery, administrative organizer management, QR-based attendance, downloadable reports, automated tests, and production deployment hardening. Cancellation and feedback remain outside the current project scope.

REFERENCES

Oracle. Java Platform Standard Edition 17 API Specification. https://docs.oracle.com/en/java/javase/17/docs/api/

Oracle. JDBC API and JDBC documentation. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

Apache Software Foundation. Apache Tomcat 9 Documentation. https://tomcat.apache.org/tomcat-9.0-doc/

Oracle. MySQL 8 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

Apache Software Foundation. Maven Documentation. https://maven.apache.org/guides/

MDN Web Docs. HTML, CSS, and JavaScript documentation. https://developer.mozilla.org/en-US/docs/Web

Eclipse Foundation. JavaMail project documentation. https://javaee.github.io/javamail/

jBCrypt. Java implementation of BCrypt password hashing. https://www.mindrot.org/projects/jBCrypt/

Campus Event Management System. Current project source code and README.md. Local project implementation.